

CSC 581 Milestone 3

Saisumanth Tallapragada
stallap@ncsu.edu

1 Engine Design

The main objective for Milestone 3 was to modify our existing framework into a scalable and flexible engine that could support my individual game "Skully Bird" and other games built using this engine. To do this, we created a property-based, modular object model, integrated it with a multithreaded network layer, and built an engine for performance comparison to assess various data sync techniques. The architecture is data-driven, with no inheritance hierarchies, allowing us for quick modifications of gaming dynamics and enabling future feature developments.

1.1 Motivation and Alternatives

The engine is built with **property-based object model** managed through a central **Registry**. Each game object, in this case the player, pipe, or a moving platform, stores its data as flexible key-value pairs using `std::variant`. This made every object more easy to modify at runtime rather than the compile time updates. Instead of relying on class inheritances or hierarchies, we could just make small updates to control behaviors like gravity (physics), movement, or camera scrolling. This design fits *Skully Bird* perfectly because almost everything in the game like players, spawn points, death zones, and pipes could be described through shared data and simple updates.

We explored other options before choosing this approach. A traditional inheritance model felt very rigid and would require creating a new class for every gameplay feature which is not flexible when adding more features in the game as we develop in future. A full ECS design was powerful but complicated for what we needed. The property-based model offered the best balance between flexibility and simplicity. It allowed us to add or modify game objects while the game was running and made networking easier since the same properties used in gameplay could be sent across clients through the `PeerManager.h` and `PeerManager.cpp`.

1.2 Capabilities and Limitations

The property-based model design made our engine very flexible and easy to expand in case of any future developments. New objects like spawn points, moving platforms, or even invisible boundaries could be created just by setting a few properties in the registry no engine changes were needed to modify our game. Because every object used the same structure, it also uses the same physics, rendering, and networking layers consistently. This allowed features like

player synchronization across multiple clients and camera scrolling to work smoothly across all clients. The system also made testing faster since we could tweak (or) add behaviors directly in gameplay code without touching the engine core.

However, this approach have a few trade-offs. Since properties are stored as strings and variants,

```
std::unordered_map<std::string, std::variant<int, float, bool, Vec2, std::string>>
```

small mistakes such as typos or wrong data types can cause runtime errors instead of compiler warnings. Accessing data through maps also adds a bit of overhead, which might matter if the game scaled to thousands of objects. For a 2D platformer like *Skully Bird*, these drawbacks were minor compared to the flexibility and the speed we acheived during the game development.

1.3 Implemented Behaviors and Properties

Using the property model based design in *Skully Bird* was built through data and simple update functions rather than hard-coded classes or inheritances. The player character, for example, is just a game object with position, velocity, and physics properties. Its behavior is controlled by a small update function that applies physics each frame and triggers a jump when the space-bar is pressed. Platforms work the same way static ones stay fixed, while moving platforms include a short movement or oscillation function that shifts their position horizontally or vertically.

Other parts of the game ENvironment, like spawn points and death zones, are invisible objects that are used when the player collides with a death zone or a boundary, they are instantly spawned back to the nearest spawn point using simple collision checks. The scrolling camera also uses this data-driven idea: instead of moving every object in the game enviornment, the engine updates a single camera offset property to shift the player's view as they move forward or backward. Even pipes and obstacles are managed this way, spawned periodically and updated through shared behaviors.

Game Controls and Features

- Move Left: A
- Move Right: D
- Jump: Space
- Pause/Unpause: P
- Slow motion: 1
- Normal speed: 2
- Double speed: 3
- Toggle scaling mode: T

- **Quit:** ESC

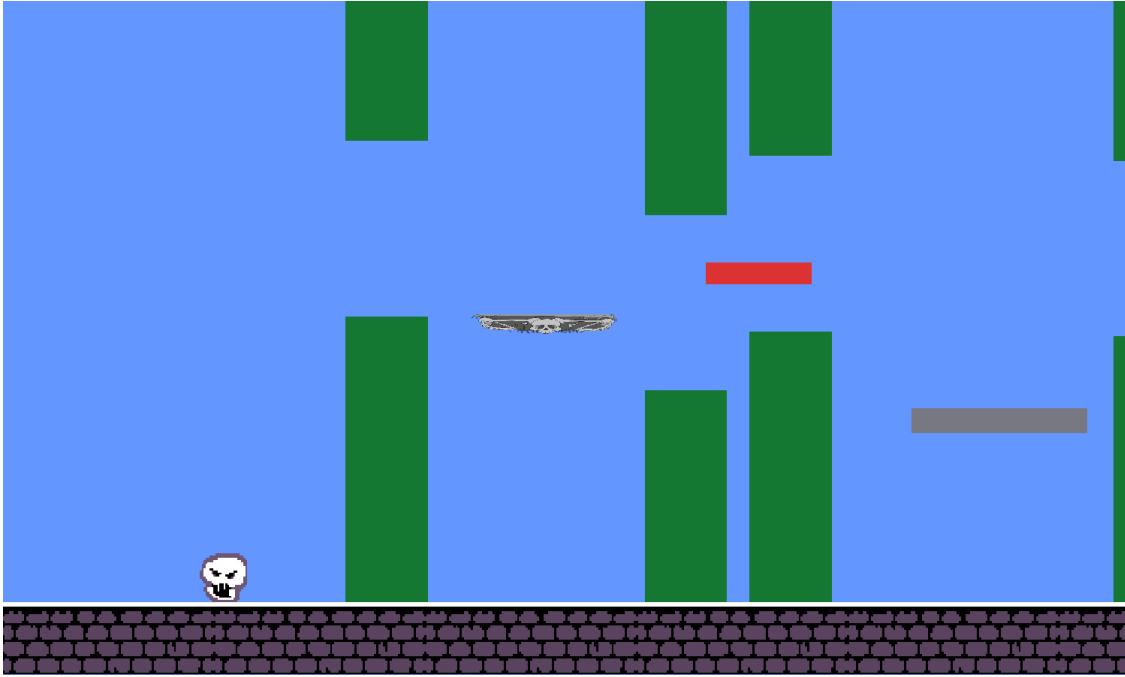


Figure 1: Here there are 3 static and 2 moving platforms

2 Network Architecture

2.1 Model Overview

Our engine is a **hybrid networking model** that combines a central server with direct peer-to-peer communication. The server, implemented in `server.cpp` using ZeroMQ’s REQ/REP sockets, maintains control over the shared world—spawning pipes, handling collisions, and enforcing the global timelines. Clients connect to this server for environmental changes while simultaneously broadcasting their own player states to other clients through a PUB/SUB network managed by the `PeerManager`.

This method maintained uniformity while keeping the games responsive. Each client separately manages local inputs and publishes its movement or pause status to peers, while the server’s asynchronous loop transmits pipe data and changes the scene constantly. Because of this, players can see other characters move almost instantly without having to wait for the server, the entire world is still controlled by the server.

2.2 Concurrency and Synchronization

Each client runs a dedicated `PeerManager` instance that manages both networks at once: one thread sends player input or position updates, while another listens for incoming data from peers and the server. A shared map of `PlayerData` objects stores every peer’s current

position, toggle or scale, and pause state, by using `std::mutex`. The server likewise spawns separate threads for handling requests and advancing the game loop, ensuring that slow or dropped clients never stall the simulation they exit the game after a 5 seconds timeout delay. Client disconnects are detected through timeouts and cleaned up automatically so that remaining players remain unaffected.

2.3 Data Formats

The engine supports two data-exchange formats that can be toggled at runtime to check the performance:

- **Full-State Mode:** Each update transmits every object's absolute position, pause state, and scale.
- **Input-Delta Mode:** Only input actions—left, right, and jump are sent.

3 Performance Analysis

3.1 Experimental Setup

We created a simple performance testing framework that was built right into the engine to evaluate the effectiveness of our two networking options. By changing the amount of clients, moving objects, and loop iterations, the tests replicated various gaming loads. To determine the total execution time in milliseconds for both network strategies—**Full-State** and **Input-Delta**—we conducted three primary tests. To compare stability and performance, each setup was run multiple times, and the average and variance were noted.

All tests were run locally. The three test setups were:

1. 2 clients, 100 objects, 100,000 iterations
2. 2 clients, 100 objects, 50,000 iterations
3. 4 clients, 500 objects, 100,000 iterations

3.2 Results and Observations

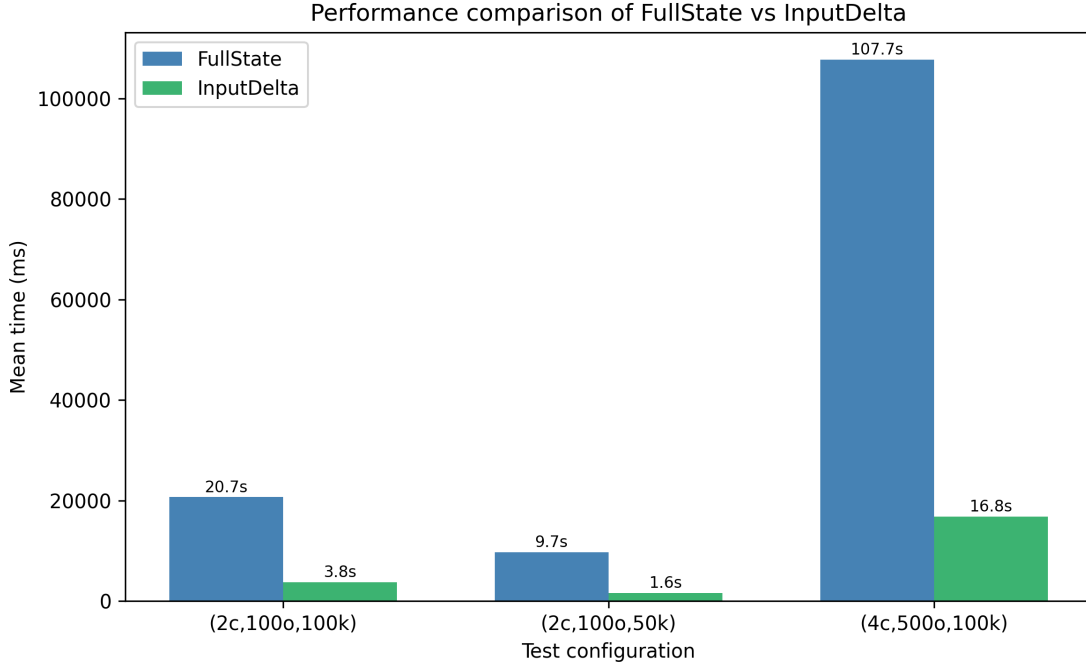


Figure 2: Comparison of mean execution times for Full-State and Input-Delta strategies.

For the smallest test (2 clients, 100 objects, 100,000 iterations), Full-State took roughly 20,672 ms on average, while Input-Delta completed the same run in only 3,753 ms—an improvement of about 82%. When we increased the object count and clients (4 clients, 500 objects), the difference became even more: Full-State averaged 107,732 ms compared to just 16,781 ms for Input-Delta, maintaining similar variance and far greater efficiency.

Message sizes and processing times increase linearly with the number of objects in the Full-State mode since each object’s whole position and state are serialised and delivered with each frame. On the other hand, only player input modifications and timestamps are transmitted in the Input-Delta mode. Regardless of the complexity of the scene, this maintains messages small and almost constant in size. As a result, the input-delta method scales considerably better, particularly when the number of clients or dynamic objects rises.

3.3 Analysis and Takeaways

These outcomes indicate that a differential updating technique works well for real-time games. Sending only input deltas keeps the gameplay consistent across clients while reducing bandwidth and CPU overhead. Even though Full-State updates ensure flawless consistency, big multiplayer scenes cannot afford them. The Input-Delta model offered steady frame timing even at greater loads and smooth gameplay with low latency for our engine and the *Skully Bird* game. Overall, the performance tests showed that our hybrid architecture and delta-based synchronization are very scalable in addition to being functionally correct. Our

design choices were strongly influenced by the analysis’s findings, which also demonstrated that our networking strategy could accommodate more participants and intricate scenarios at a relatively low additional cost.

4 Reflection

This milestone expanded the engine and presented a number of additional obstacles. Compilation problems occurred when the new object model was integrated with the current client-server code, particularly with structured bindings in `server.cpp`, necessitating registry updates to enable iteration. Networking problems like stale peer connections and desynchronized updates were common at initially, and tweaking collision detection on moving platforms required numerous test runs to get right.

The engine became much more cohesive and stable when these issues were fixed. The hybrid networking layer operated seamlessly across numerous clients with uniform frame timing, and the property-based object model greatly simplified the process of adding new entities such as spawn points and platforms. All things considered, I learned a lot about debugging intricate, multi-threaded systems and how thoughtful design choices can significantly impact stability and performance over the long run.

5 Appendix

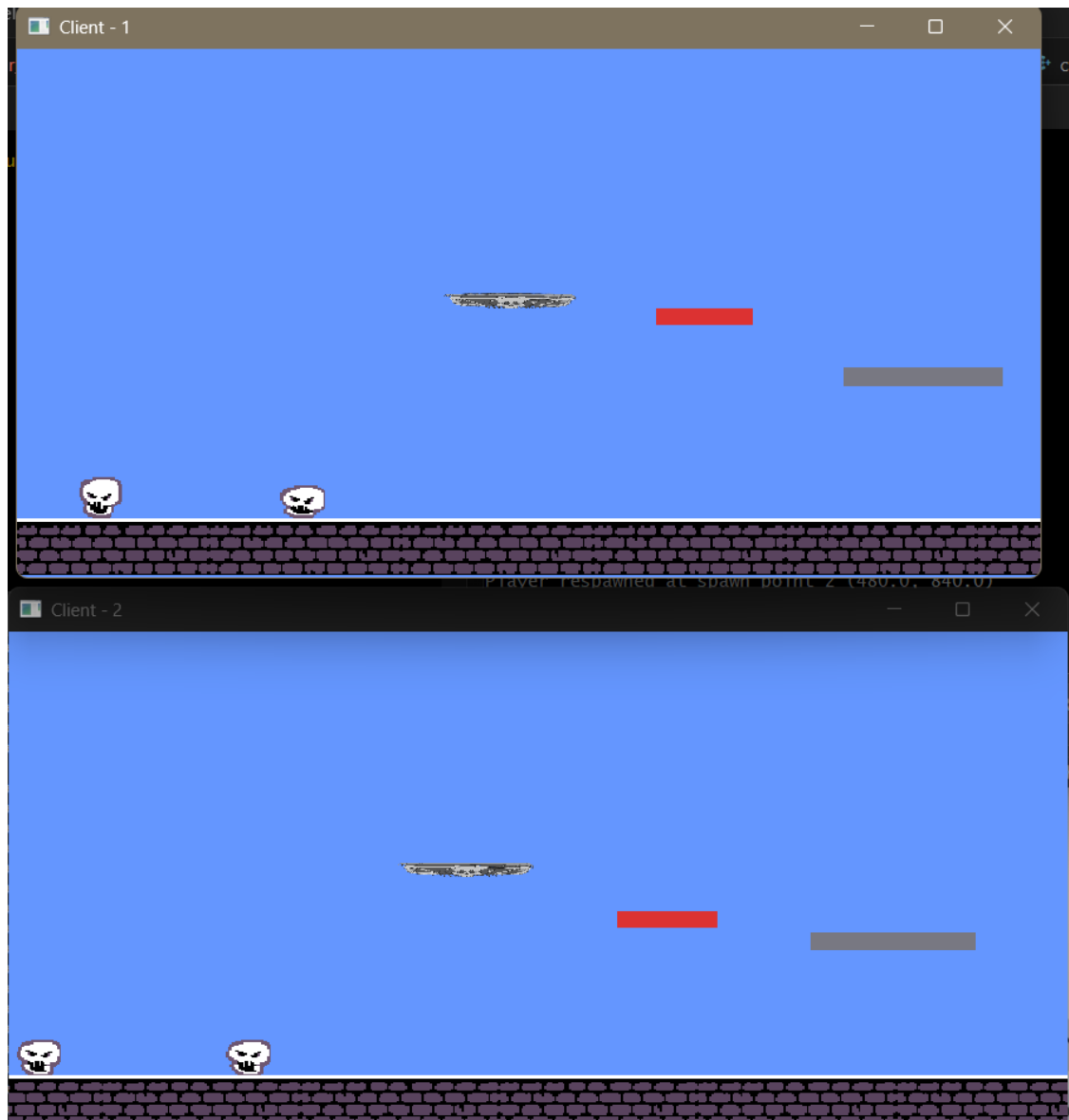


Figure 3: Gameplay showing the players, static platforms, and moving platform.

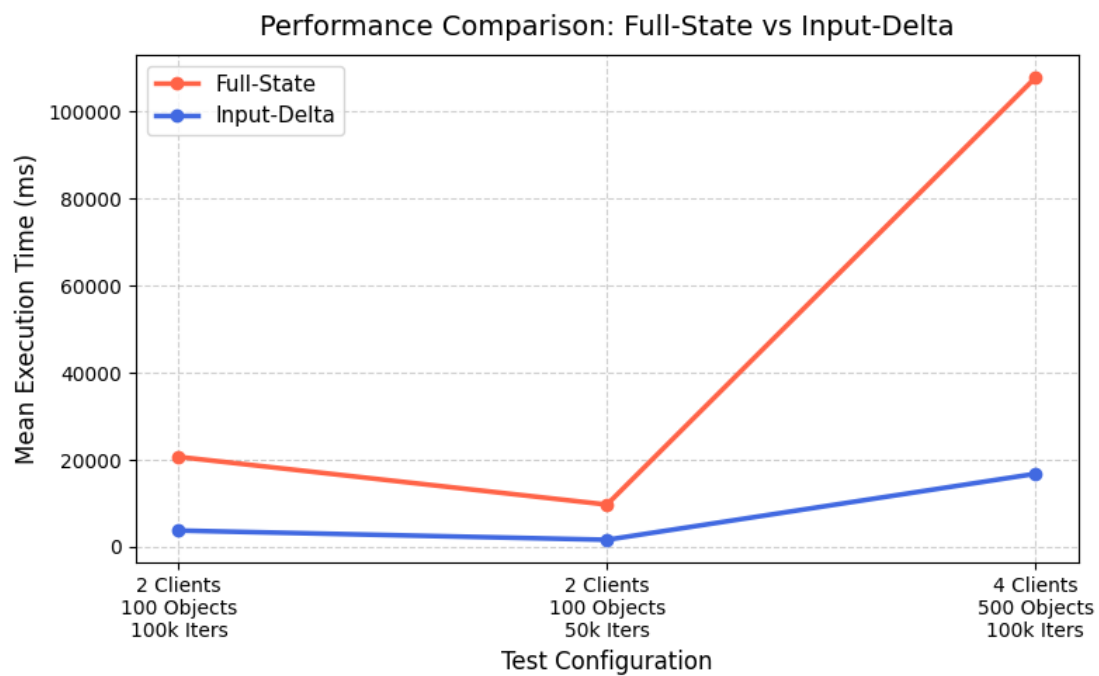


Figure 4: Performance gap clearly visible between full state and input delta